EmoSense

Face Emotion Recognition System

A Deep-Learning two-stage pipeline using YOLOv8 + CNN-based classification
trained on the RAF-DB dataset to detect 7 human emotions in real time.

CNN | YOLOv8 | Transfer Learning | RAF-DB | ONNX

Created by Hamza Khan
AI Course — Lab Presentation 2025

SECTION 01

System Overview — What Is EmoSense?

EmoSense is a real-time Face Emotion Recognition (FER) system. It points a webcam at a person, automatically locates their face in the frame, and then classifies which one of **7 emotions** — Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise — that face is showing. The entire process runs on every single frame, roughly 33 times per second.

The Two-Stage Pipeline

The system is called a two-stage pipeline because every frame goes through exactly two neural networks in sequence, one after the other:

Stage	Model	Task	Output
1	YOLOv8n-face.pt	Face Detection	Bounding box + confidence score around each face
2	YOLOv8s-cls (custom trained)	Emotion Classification	Probability scores for all 7 emotion classes

> **Key Insight**
>
> Splitting the problem into two stages is smarter than trying to do everything in one model. Face detection is a well-solved problem — we use a pre-trained specialist for it. The emotion classifier then only ever sees a tightly cropped face, never the whole messy background. This division of labour makes both models smaller, faster, and more accurate.

SECTION 02

What Is a CNN? — The Core AI Concept

A **Convolutional Neural Network (CNN)** is a type of deep learning model specifically designed to process visual data — images and video. It is the dominant AI architecture for computer vision and is the engine powering both stages of EmoSense.

Why a Normal Neural Network Fails on Images

A regular (fully connected) neural network treats an image as a flat list of pixel values. A small 224x224 colour image has 224 × 224 × 3 = 150,528 individual numbers. Connecting every pixel to every neuron in the next layer would require hundreds of millions of parameters just in the first layer — impossible to train efficiently and prone to overfitting. More importantly, it loses all spatial structure: it no longer knows that pixels next to each other are related.

How CNNs Solve This — Three Key Operations

1. Convolutional Layers (Feature Detection)

A convolution slides a small grid of numbers called a **filter** or **kernel** (typically 3×3 or 5×5 pixels) across the entire image. At each position it multiplies each pixel under the filter by the corresponding filter value and sums them all up, producing a single output number. This produces a **feature map** — a new image that highlights wherever the pattern the filter was looking for appeared in the original image. Early layers learn simple patterns like edges and colour gradients. Deeper layers combine these to detect complex structures like eyes, noses, and ultimately whole facial expressions.

2. Pooling Layers (Spatial Compression)

After convolution, a pooling layer shrinks the feature map by replacing each small region (e.g. 2×2 pixels) with just one value — usually the maximum (Max Pooling). This reduces the amount of data, makes the network less sensitive to exact pixel positions (so a smile is still a smile whether it is 3 pixels left or right), and controls overfitting.

3. Fully Connected Layers (Final Classification)

After several rounds of convolution and pooling, the network has extracted rich, abstract feature representations. These are flattened into a 1D vector and passed through one or more fully connected layers — like a standard neural network — which combine all the features and produce one output score per class. A **Softmax** function converts these scores into probabilities that sum to 1.0, so the model outputs something like: Happy 72%, Neutral 15%, Surprise 8%, others <5%.

Layer Type	What It Does	What It Learns in EmoSense
Conv Layer 1	Applies 3×3 filters across the image	Edges, colour contrasts, lighting gradients
Conv Layer 2-3	Deeper feature extraction	Eyebrows, mouth curves, wrinkle patterns
Deep Conv Layers	High-level combination features	Full facial expressions, muscle groups
Pooling Layers	Shrink feature maps 2x	Position-invariant face features
Fully Connected	Combine all features	Which combination = which emotion
Softmax Output	Convert to probabilities	7 emotion probabilities summing to 100%

> **Why CNNs Are Perfect for Emotion Recognition**
>
> Human emotions are expressed through very specific, localised facial features — the curl of the lips, the furrow of the brow, the widening of the eyes. CNNs are uniquely suited to detect these because their convolutional filters learn to respond to exactly those small, spatially-defined patterns. A flat neural network would lose the spatial arrangement of features entirely.

SECTION 03

CNNs Applied In This Project — In Depth

EmoSense does not implement a CNN from scratch. Instead it uses **YOLOv8** (You Only Look Once, version 8), which is built on top of a sophisticated CNN backbone. Understanding how YOLO uses CNNs is key to answering technical questions in your presentation.

Stage 1 — YOLOv8n-face (Face Detection CNN)

YOLOv8n-face is a specialised object-detection network. Its CNN backbone processes the entire camera frame (resized to 640×640 pixels) in one forward pass. The CNN extracts feature maps at multiple scales — important because faces can be large (close to camera) or small (far away). The detection head then predicts: where each face is (x, y, width, height of the bounding box) and how confident it is (a score from 0 to 1).

After the CNN produces many candidate boxes, **Non-Maximum Suppression (NMS)** is applied: boxes are ranked by confidence, the best one is kept, and any other box overlapping it by more than 40% (measured as Intersection over Union, IoU) is deleted. This removes duplicate detections of the same face.

The detected box is then **tightened** before passing to Stage 2: it is made square, and trimmed 4% on the sides, 2% from the top, and 10% from the bottom (to cut out the neck and shoulders), leaving only the head region.

Stage 2 — YOLOv8s-cls (Emotion Classification CNN)

This is the model you trained yourself. It is a **classification CNN** — it takes a single 224×224 pixel image (the cropped face) and outputs 7 probabilities, one per emotion class. The 's' in YOLOv8s means 'small' — it has more parameters than the 'nano' variant, giving significantly better accuracy on a 7-class problem with a negligible speed penalty on an RTX 3070.

The CNN backbone of YOLOv8s-cls is based on **CSP (Cross Stage Partial) bottleneck blocks**. Each block splits the feature map into two paths — one goes through several convolutional layers, the other goes through directly — and the results are concatenated. This design gives the network a better gradient flow during training (gradients can flow directly backwards without passing through the convolution layers) which allows it to be trained deeper and faster with less memory.

Transfer Learning — Pre-trained Weights

Transfer learning is one of the most important techniques in modern AI. Instead of initialising the CNN's weights randomly and training from scratch (which would require millions of images and days of compute), EmoSense starts from **yolov8s-cls.pt** — a model that has already been trained on ImageNet, a dataset of 1.2 million diverse images across 1000 categories.

The pre-trained model already knows how to detect low-level features (edges, textures, colours) and mid-level features (shapes, object parts). Fine-tuning on RAF-DB then teaches the final layers how to combine those already-learned features into the 7 emotion-specific patterns. This is why training takes 30–60 minutes rather than days.

Transfer Learning Analogy

Imagine hiring an experienced artist to learn a new painting style. They already know how to mix colours, control brush pressure, and understand light and shadow — all of which took years to learn. Teaching them the new style takes a few weeks, not years. Transfer learning works the same way: the CNN already knows how to 'see'; it just needs to learn what 'angry' vs 'happy' looks like.

SECTION 04

The Dataset — RAF-DB (Real-world Affective Faces)

The **Real-world Affective Faces Database (RAF-DB)** is one of the most respected and widely-used datasets in the emotion recognition research community. It was collected from the internet under real-world conditions — not in a laboratory — which makes it significantly harder and more representative of actual use cases than earlier lab-controlled datasets like CK+ or JAFFE.

Dataset Statistics

Emotion	Folder	Train Images	Test Images	Notes
Happy	4	~4,772	~1,185	Most common class — smiling is universal
Neutral	7	~2,524	~680	Second most common
Sad	5	~1,982	~478	Often confused with neutral
Angry	6	~705	~162	Visible in brow tension and jaw
Surprise	1	~1,290	~329	Wide eyes, open mouth
Fear	2	~281	~74	Rarest — very hard to express naturally
Disgust	3	~717	~160	Nose wrinkle, lip curl
TOTAL	**—**	**~12,271**	**~3,068**	**~15,339 total labelled images**

Why RAF-DB Is Excellent

- **Real-world conditions:** Images were collected from the internet — varied lighting, angles, ethnicities, ages, and occlusions. The model must generalise to reality, not a sterile lab.
- **Expert annotation:** Each image was labelled by 40 crowdworkers and then a team of trained annotators resolved disagreements. This makes the labels highly reliable.
- **Size:** Over 15,000 images across 7 classes is sufficient to fine-tune a pre-trained CNN without overfitting.
- **Class diversity:** Having both common (Happy, Neutral) and rare (Fear, Disgust) classes forces the model to learn subtle distinguishing features.

Class Imbalance — The Key Challenge

RAF-DB has a severe class imbalance: Happy has ~17× more images than Fear. If the model just predicted 'happy' for every face, it would be correct for a large portion of samples without ever learning anything useful. The training script counters this with **data augmentation** — artificially creating more variety in the minority classes.

SECTION 05

Training the Model — How the CNN Learns

The Training Loop — 80 Epochs on an RTX 3070

Training was run for **80 epochs** on an **NVIDIA RTX 3070 GPU** (8 GB VRAM). One epoch is one complete pass through all ~12,271 training images. At a batch size of 64, each epoch requires roughly 192 gradient update steps. Total training time was approximately **30–60 minutes** — made possible by GPU acceleration and transfer learning.

What Happens in Each Step

Step 1: Forward Pass

The batch of face images is pushed through all CNN layers. Each image produces 7 probability scores — one per emotion.

Step 2: Loss Calculation

The predicted probabilities are compared to the correct labels using Cross-Entropy Loss. If the model predicted 'neutral' with 90% confidence but the correct answer was 'angry', the loss is high. A perfect prediction gives near-zero loss.

Step 3: Backward Pass (Backpropagation)

The loss value is differentiated with respect to every single weight in the CNN using the chain rule of calculus. This tells us exactly how much each weight contributed to the error.

Step 4: Weight Update (AdamW Optimizer)

The AdamW optimizer adjusts each weight slightly in the direction that reduces the loss. It uses adaptive learning rates per weight — weights that have been consistently wrong get larger updates.

Step 5: Learning Rate Schedule

The learning rate starts at 0.001, warms up gradually for the first 3 epochs (to avoid unstable early updates), then decays toward 0.01 over the remaining epochs following a cosine schedule.

Step 6: Early Stopping

If validation accuracy does not improve for 20 consecutive epochs, training halts automatically. This prevents overfitting and saves GPU time.

Data Augmentation — Teaching Generalisation

Before each image is fed to the model during training, it is randomly modified by augmentation transforms. The label stays the same — a happy face is still happy after being flipped — but the model sees a slightly different version each epoch, preventing it from simply memorising the training images.

Augmentation	What It Does	Why It Helps
HSV Jitter	Randomly shifts hue, saturation, brightness	Handles varied lighting and skin tones
Random Rotation (±15°)	Tilts the face image slightly	Handles tilted or bowed heads

Translation	Shifts image up/down/left/right	Handles off-centre face crops
Random Zoom	Scales in slightly	Handles varying distances from camera
Horizontal Flip	Mirrors the face left-right	Faces look the same mirrored; doubles dataset size
Random Erasing	Randomly blacks out small patches	Forces model to use whole face, not one feature
Mixup	Blends two images together with weighted label	Improves calibration on ambiguous expressions

Model Export to ONNX

After training, the model is exported from PyTorch format (.pt) to **ONNX format (.onnx)**. ONNX (Open Neural Network Exchange) is a universal model format that can run on different hardware backends without needing PyTorch installed. The app uses **ONNX Runtime with GPU** for inference, which is faster than standard PyTorch because it performs graph-level optimisations at load time — fusing operations, eliminating redundant computations, and taking advantage of the RTX 3070's Tensor Cores.

SECTION 06

The Application — Real-Time Detection

Architecture of the App

The app is built with **CustomTkinter** — a modern Python GUI library. The window has three panels: a title bar at the top, the live camera feed on the left, and a control sidebar on the right. Every 30 milliseconds the app grabs a new frame from the webcam and passes it through the full two-stage detection pipeline.

The Confidence Threshold Slider

The sidebar includes a **confidence threshold slider**. When the emotion classifier produces its 7 probability scores, it picks the highest one as the prediction. If that highest probability is below the threshold, the detection is discarded and no label is drawn on that face.

Why Do We Need a Confidence Threshold?

Scenario	Without Threshold	With Threshold
Neutral face, poor lighting	Labels it 'sad' at 32% confidence	Suppressed — not displayed (threshold = 50%)
Clearly happy face	Labels 'happy' at 89% confidence	Displayed — passes threshold easily
Ambiguous expression	Labels 'fear' at 41% — may be wrong	Suppressed — user not misled

> **Why This Matters for AI Systems**
>
> Emotion recognition is inherently ambiguous — two people can look at the same face and disagree. A well-designed AI system should communicate its uncertainty rather than always confidently outputting an answer. The confidence threshold is a simple but effective mechanism for controlling the precision vs. recall trade-off: higher threshold = fewer but more accurate labels. This is a fundamental concept in machine learning that applies far beyond this project.

Emotion Statistics Panel

The sidebar maintains a running tally of every emotion detected since the camera started. After each frame with at least one valid detection, these counts are normalised into percentages and displayed as progress bars. The **dominant emotion** — the one with the highest confidence in the current frame — is shown as a large emoji at the bottom. This gives a real-time aggregate view of emotional state over time.

SECTION 07

Key Design Decisions — The 'Why' Behind the Choices

Why 224×224 pixels?

224×224 is the ImageNet standard input size — the dataset that most CNNs (including YOLOv8s-cls) were originally trained on. It is large enough to capture all facial detail needed for emotion recognition (eyes, mouth, brow) but small enough to process at real-time speeds. Using a different size would break compatibility with the pre-trained weights.

Why YOLOv8s-cls over YOLOv8n-cls?

The 's' (small) variant has significantly more parameters than 'n' (nano), which gives meaningfully better classification accuracy on a 7-class problem. On an RTX 3070 the inference time difference is only a few milliseconds — negligible. Accuracy improvement is worth the marginal cost.

Why ONNX Runtime over standard PyTorch for inference?

ONNX Runtime performs graph-level optimisations when loading the model — fusing layers, eliminating dead branches, choosing the fastest kernel for the GPU. Standard PyTorch executes operations one by one. For a fixed-input-size inference task like this, ONNX is measurably faster with no accuracy loss.

Why a separate face detector (Stage 1)?

Feeding the entire camera frame to the emotion classifier would force it to handle backgrounds, multiple people, and varying face sizes — dramatically increasing task complexity. By first detecting and tightly cropping the face, the emotion classifier always receives a standardised, face-only 224×224 image. This separation of concerns improves both components.

Why 80 training epochs?

80 epochs provides a good balance between training time (~30-60 min on RTX 3070) and reaching convergence. The early stopping mechanism (patience = 20 epochs) means the model automatically stops if it plateaus before epoch 80, preventing overfitting while ensuring thorough training.

Why AdamW instead of SGD?

AdamW is an adaptive optimizer that maintains separate learning rates for each parameter and includes weight decay decoupled from the gradient update. This makes it much faster to converge than vanilla SGD on classification tasks, especially with transfer learning where different layers may need different update rates.

SECTION 08

Lab Q&A; Preparation — Questions You May Be Asked

The following are likely questions from your examiner, with strong answers prepared.

Q: What type of AI is this?

A: This is Supervised Deep Learning — specifically a Convolutional Neural Network (CNN). It is supervised because every training image has a human-provided label. It is deep learning because the CNN has many stacked layers. It is a CNN because it uses convolutional filters to detect spatial patterns in image data.

Q: What is the difference between the two stages?

A: Stage 1 is object detection — it finds WHERE the faces are in the image. Stage 2 is image classification — it determines WHAT emotion is on each detected face. Detection and classification are two different computer vision tasks requiring different model architectures.

Q: What is Transfer Learning and why did you use it?

A: Transfer learning reuses a model trained on a large general dataset (ImageNet) as the starting point for training on a smaller specialised dataset (RAF-DB). The pre-trained model already knows how to detect edges, textures, and shapes — universal visual features useful for any image task. We only need to teach the final layers what 'angry' vs 'happy' looks like, not how to see.

Q: What is overfitting and how did you prevent it?

A: Overfitting is when a model memorises the training data instead of learning generalisable patterns. On new (unseen) images it performs poorly. EmoSense prevents overfitting with: (1) data augmentation — each image looks slightly different every epoch, (2) early stopping — training halts when validation accuracy stops improving, (3) weight decay via AdamW — penalises large weights.

Q: Why does the model sometimes get the emotion wrong?

A: Emotion recognition is genuinely hard — even humans disagree on ambiguous expressions. The challenges include: class imbalance (fear and disgust have far fewer training examples), cross-cultural differences in expression, occlusions (hair covering face), poor lighting, and the fact that some expressions — like sad and neutral — look very similar. The confidence threshold slider helps by suppressing low-confidence predictions.

Q: What is the role of the GPU in training?

A: Training a CNN requires billions of floating-point multiplications per epoch. A CPU does these sequentially; an NVIDIA RTX 3070 GPU has 5,888 CUDA cores that do them in parallel — making training roughly 50-100x faster. Without a GPU, 80 epochs on RAF-DB would take many hours; with the RTX 3070 it takes 30-60 minutes.

Q: What is ONNX and why export to it?

A: ONNX (Open Neural Network Exchange) is a universal model format. ONNX Runtime is an inference engine that performs graph optimisations at load time — fusing operations and eliminating redundant computations — then runs the optimised graph using GPU Tensor Cores. This makes inference faster than standard PyTorch for a fixed-input-size application like real-time webcam emotion detection.

Q: What is Non-Maximum Suppression?

A: NMS is a post-processing step after Stage 1 (face detection). The CNN may produce multiple overlapping boxes for the same face. NMS ranks them by confidence, keeps the highest-confidence box, then removes any other box that overlaps it by more than 40% (IoU threshold). This ensures each face gets exactly one bounding box.

Q: What does Softmax do?

A: Softmax is the final activation function in the classification CNN. The network outputs 7 raw scores (called logits) — one per emotion. Softmax converts these into probabilities by exponentiating each score and dividing by the sum of all exponentiated scores. The result is 7 numbers that are all between 0 and 1 and sum to exactly 1.0 — a proper probability distribution.

Q: Could this system be biased?

A: Yes — all AI systems can be biased. RAF-DB was collected from internet images which may over-represent certain ethnicities or age groups. If the training data is skewed, the model may perform better on some demographic groups than others. This is a known limitation of the dataset and an active area of research in fair AI.

SECTION 09

Glossary — Terms to Know Cold

CNN (Convolutional Neural Network)

A deep learning model that uses sliding filter kernels to detect spatial patterns in images. The dominant architecture in computer vision.

YOLOv8

You Only Look Once v8 — a state-of-the-art real-time object detection and classification framework built on CNN backbones.

Transfer Learning

Reusing a model pre-trained on a large dataset as the starting point for training on a smaller, more specific dataset.

Fine-tuning

The process of continuing to train a pre-trained model on new data, updating weights to specialise for the new task.

Epoch

One complete pass through the entire training dataset.

Batch Size

The number of images processed together before one gradient update. EmoSense uses batch size 64.

Loss Function

A mathematical measure of how wrong the model's predictions are. Training minimises this. EmoSense uses Cross-Entropy Loss.

Backpropagation

The algorithm that calculates how much each weight in the network contributed to the loss, enabling the optimizer to update them.

AdamW

An adaptive optimizer that maintains per-parameter learning rates and decouples weight decay from the gradient update.

Learning Rate

How large each weight update step is. Too high = unstable training. Too low = too slow.

Overfitting

When a model memorises training data and performs poorly on new, unseen examples.

Data Augmentation

Artificially creating varied versions of training images (flips, rotations, colour shifts) to improve generalisation.

Softmax

Activation function that converts raw CNN output scores into a probability distribution summing to 1.0.

Confidence Score

The probability value assigned to the top predicted class. High = model is sure. Low = model is uncertain.

Non-Maximum Suppression (NMS)

Post-processing that removes duplicate bounding boxes, keeping only the highest-confidence one per face.

IoU (Intersection over Union)

A measure of how much two bounding boxes overlap. Used in NMS to decide which boxes are duplicates.

ONNX

Open Neural Network Exchange — a universal model format enabling efficient cross-platform inference.

Bounding Box

A rectangle drawn around a detected object (face) defined by x, y, width, height coordinates.

RAF-DB

Real-world Affective Faces Database — ~15,000 labelled face images from the internet covering 7 basic emotions.

Class Imbalance

When some classes have far more training examples than others. Can bias the model toward majority classes.

Precision vs Recall

Precision = fraction of model's positive predictions that are correct. Recall = fraction of true positives the model detected. The confidence threshold controls this trade-off.

Feature Map

The output of a convolutional layer — a transformed version of the input image highlighting where specific patterns appear.

CSP Bottleneck

Cross Stage Partial — an architectural design in YOLOv8 that splits feature maps into two paths for better gradient flow.